

2025 Dev@Source



Random Talk 01

λ Calculation

代入 演算

Presenter: Epi

Section 1

alpha, beta, and lambda

α Conversion & λ -abstract

- 😍 → 😍 ➡

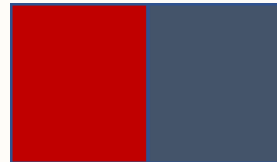
- ★ → ★ ➡

- 🚫 → 🚫 ➡

■ → ■ ➡

β Reduction

- (   ) let  = 
-  



λ Calculation

- α Conversion
 - That's a simple name conversion.
 - β Reduction
 - Takes a input to reduce a variable.
- $\lambda x.M$
 - $(\lambda x.M)N$
 - $M[N/x]$

Conclusion

Remember those sentences and they are helpful !



Church–Turing thesis

λ Calculation can make **ANYTHING** a computer can do



First-Class Functions

Function can be passed as **inputs** to functions

Function can return functions as an **output**



Everything is **function** (in specific perspective).

$$\text{True} = \lambda x. \lambda y. x$$

$$\text{False} = \lambda x. \lambda y. y$$

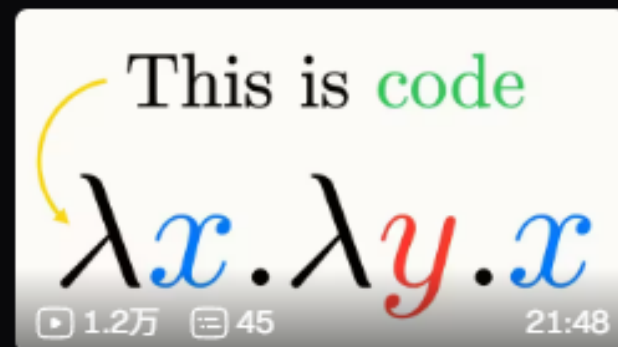
$$3 = \lambda f. \lambda x. f(f(f\ x))$$

$$Y = \lambda f. (\lambda x. f(x\ x)) (\lambda x. f(x\ x))$$

$$\text{pair} = \lambda x. \lambda y. \lambda p. p\ x\ y$$



加号X加号?【lambda演算】



λ 演算之美

Example 1

Lets solve a simple (?) problem

Example

- 将下两行代入甲、乙：「甲；「甲；乙」」；
- 将下两行代入甲、乙：「甲；「甲；乙」」；
- 将本段演算中用于提取的数字加一；
- 得到数字：0

Example: 分层

- 将下两行代入甲、乙：「甲；「甲；乙」」；
 - 将下两行代入甲、乙：「甲；「甲；乙」」；
 - 将本段演算中用于提取的数字加一；
- 得到数字：0

Example: 分层

- 将下两行代入甲、乙: 「甲: 「甲; 乙」」;
- 将下两行代入甲、乙: 「甲: 「甲; 「甲; 乙」」」;
- 将本段演算中用于提取的数字加一;
- 得到数字: 0

一个显然的洞见是，第一次代入前面的几行一定会构成一个函数，然后这个函数作用到0上。

所以我们先关注函数本身

Example 第一次代入

- 将下两行代入甲、乙：「甲；「甲；乙」」；
 - 将下两行代入甲、乙：「甲；「甲；乙」」；
 - 将本段演算中用于提取的数字加一；

我知道你很急，但是你先别急。请仔细想一想：

- 1) 谁是甲，谁是乙。（而且带不带最后的分号？）
- 2) 代入后会得到的东西的具体结构（比如带不带最外层括号？）

Example 第一次代入

目标形式

- 将下两行代入甲、乙: 「甲; 「甲; 乙」」;
- 将下两行代入甲、乙. 甲 「甲; 「甲; 乙」」;
- 将本段演算中用于乙提取的数字加一;

Example 第一次代入

目标形式

甲; 「甲; 乙」

将下两行代入甲、乙: 甲「甲; 「甲; 乙」」;

将本段演算中用于乙提取的数字加一

Example 第一次代入

目标形式

甲; 「甲; 乙」

将下两行代入甲、乙: 「甲; 「甲; 乙」」 ;

「 将下两行代入甲、乙: 「甲; 「甲; 乙」」 ; 将本段演算中用于提取的数字加一 」

Example 第一次代入后

- 将下两行代入甲、乙：「甲；「甲；乙」」；
- 「将下两行代入甲、乙：「甲；「甲；乙」」；将本段演算中用于提取的数字加一」；
- 得到数字：0

为了方便，我们把

- “将本段演算中用于提取的数字加一” 记作：Succ()

Rethink

Lets find a better solution for solving

文字

- 将下两行代入甲、乙：「甲；「甲；乙」」；
- 将下两行代入甲、乙：「甲；「甲；乙」」；
- Succ()



- 将下两行代入甲、乙：「甲；「甲；乙」」；
- 「将下两行代入甲、乙：「甲；「甲；乙」」；Succ()」；
- 0

Function takes **Function** as input and return a **Function**

一般记法

• function Calc (function a, function b) that returns function a (a, b)

- Let a = Calc ()
- Let b = Succ ()

→ ?

• 将下两行代入甲、乙: 「甲; 「甲; 乙」」;

- 将下两行代入甲、乙: 「甲; 「甲; 乙」」;
- Succ ()
- Zero ()



- 将下两行代入甲、乙: 「甲; 「甲; 乙」」;
- 「将下两行代入甲、乙: 「甲; 「甲; 乙」」; ; Succ ()」;
- Zero ()

- 好难表记, 事情变得困难了!
- 普通的函数记法在涉及到函数自指的时候显得比较神秘。
- Calc 有的时候传函数有的时候传值, 好无力

Function takes Function as input and return a Function

Lambda 记法

$\lambda a. \lambda b. a(a(b))$

• $\lambda a. \lambda b. a(a(b))$

• $\lambda x. x+1$

• $\lambda x. 0$

• 将下两行代入甲、乙：「甲；「甲；乙」」；

• 将下两行代入甲、乙：「甲；「甲；乙」」；

• 将本段演算中用于提取的数字加一；

• 得到数字：0

Lambda function 让表记变得清晰了！

Lambda function 有良好的对应关系！

Lambda 记法

$\lambda a. \lambda b. a(a(b))$

• $\lambda a. \lambda b. a(a(b))$

• $\lambda x. x+1$

• $\lambda x. 0$

• $(a \Rightarrow b \Rightarrow a(a(b)))$

• $(a \Rightarrow b \Rightarrow a(a(b)))$

• $(x \Rightarrow x + 1)$

• (0)

> $(a \Rightarrow b \Rightarrow a(a(b)))(a \Rightarrow b \Rightarrow a(a(b)))(x \Rightarrow x + 1)(0)$

High level

今北産業

今来たから三行で説明して

不讲人话版本

- 将下两行代入甲、乙：「甲；「甲；乙」」；
- 将下两行代入甲、乙：「甲；「甲；乙」」；
- 将本段演算中用于提取的数字加一；
- 得到数字：0

略通人性版本

- 将下一行的事情做两遍之于再下一行的函数
- 将下一行的事情做两遍之于再下一行的函数
- 将提取的数字加一;
- 提取 0

人话版本

- 将下一行的事情做四遍
- 将提取的数字加一;
- 提取 0
- 答案是 4
- 太棒了你已经学会了lambda演算!
- 那么我们来做一些练习吧!

Code time

Let's write a simulator for this language